

DA5300 — LAB SOLUTIONS EXAM CRAM

Core implementations to understand, then re-type in Jupyter

Use this differently tonight: read the solution, understand the pattern, close it, then re-type it in a blank Jupyter cell. Do not try to invent everything from scratch the night before the exam.

1. Stack + Queue

1. Stack ADT

```
class Stack:
    def __init__(self):
        self.data = []

    def push(self, x):
        self.data.append(x)

    def pop(self):
        if self.is_empty():
            raise IndexError("Stack is empty")
        return self.data.pop()

    def top(self):
        if self.is_empty():
            raise IndexError("Stack is empty")
        return self.data[-1]

    def is_empty(self):
        return len(self.data) == 0

    def __len__(self):
        return len(self.data)
```

2. Queue ADT — simple lab implementation

```
class Queue:
    def __init__(self):
        self.data = []

    def enqueue(self, x):
        self.data.append(x)

    def dequeue(self):
        if self.is_empty():
            raise IndexError("Queue is empty")
        return self.data.pop(0)

    def first(self):
        if self.is_empty():
            raise IndexError("Queue is empty")
        return self.data[0]

    def is_empty(self):
        return len(self.data) == 0

    def __len__(self):
        return len(self.data)
```

3. Balanced brackets

```
def balanced_brackets(s):
    stack = []
    match = {'(': ')', '[': ']', '{': '}'

    for ch in s:
        if ch in '([{':
            stack.append(ch)
        elif ch in ')]}':
            if not stack or stack.pop() != match[ch]:
                return False

    return len(stack) == 0
```

4. Reverse a queue using a stack

```
def reverse_queue(Q):
    S = Stack()
    while not Q.is_empty():
```

```
        S.push(Q.dequeue())
while not S.is_empty():
    Q.enqueue(S.pop())
return Q
```

2. Linked Lists

5. Singly linked list: node + append + display

```
class Node:
    def __init__(self, element, next=None):
        self.element = element
        self.next = next

class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def is_empty(self):
        return self.head is None

    def add_first(self, x):
        self.head = Node(x, self.head)

    def add_last(self, x):
        new = Node(x)
        if self.head is None:
            self.head = new
            return
        cur = self.head
        while cur.next:
            cur = cur.next
        cur.next = new

    def display(self):
        cur = self.head
        while cur:
            print(cur.element, end=' ')
            cur = cur.next
        print()
```

6. Search + delete first occurrence

```
def search(head, x):
    cur = head
    while cur:
        if cur.element == x:
            return True
        cur = cur.next
    return False

def delete_first(head, x):
    if head is None:
        return None

    if head.element == x:
        return head.next

    cur = head
    while cur.next and cur.next.element != x:
        cur = cur.next

    if cur.next:
        cur.next = cur.next.next
    return head
```

7. Delete all occurrences

```
def delete_all(head, x):
    while head and head.element == x:
        head = head.next

    cur = head
    while cur and cur.next:
        if cur.next.element == x:
            cur.next = cur.next.next
        else:
            cur = cur.next
    return head
```

8. Reverse singly linked list — memorize this pattern

```
def reverse_linked_list(head):
    prev = None
    cur = head

    while cur:
```

```

    nxt = cur.next
    cur.next = prev
    prev = cur
    cur = nxt

    return prev

```

9. Doubly linked list: add first/last

```

class DNode:
    def __init__(self, element, prev=None, next=None):
        self.element = element
        self.prev = prev
        self.next = next

class DoublyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None

    def add_first(self, x):
        new = DNode(x, None, self.head)
        if self.head:
            self.head.prev = new
        else:
            self.tail = new
        self.head = new

    def add_last(self, x):
        new = DNode(x, self.tail, None)
        if self.tail:
            self.tail.next = new
        else:
            self.head = new
        self.tail = new

```

10. Doubly linked list: delete known node

```

def delete_node(self, node):
    if node.prev:
        node.prev.next = node.next
    else:
        self.head = node.next

    if node.next:
        node.next.prev = node.prev
    else:
        self.tail = node.prev

```

11. Positional List core

```

class Position:
    def __init__(self, container, node):
        self.container = container
        self.node = node

    def element(self):
        return self.node.element

class PNode:
    def __init__(self, e, prev=None, next=None):
        self.element = e
        self.prev = prev
        self.next = next
        self.deleted = False

class PositionalList:
    def __init__(self):
        self.head = None
        self.tail = None
        self.size = 0

    def _validate(self, p):
        if p is None or p.container is not self or p.node.deleted:
            raise ValueError('invalid position')
        return p.node

    def _pos(self, node):
        return None if node is None else Position(self, node)

    def first(self): return self._pos(self.head)
    def last(self): return self._pos(self.tail)

```

```

def before(self, p):
    return self._pos(self._validate(p).prev)

def after(self, p):
    return self._pos(self._validate(p).next)

def is_empty(self): return self.size == 0
def __len__(self): return self.size

def add_first(self, e):
    return self._insert(e, None, self.head)

def add_last(self, e):
    return self._insert(e, self.tail, None)

def _insert(self, e, pred, succ):
    n = PNode(e, pred, succ)
    if pred: pred.next = n
    else: self.head = n
    if succ: succ.prev = n
    else: self.tail = n
    self.size += 1
    return Position(self, n)

def add_before(self, p, e):
    n = self._validate(p)
    return self._insert(e, n.prev, n)

def add_after(self, p, e):
    n = self._validate(p)
    return self._insert(e, n, n.next)

def delete(self, p):
    n = self._validate(p)
    if n.prev: n.prev.next = n.next
    else: self.head = n.next
    if n.next: n.next.prev = n.prev
    else: self.tail = n.prev
    self.size -= 1
    n.deleted = True
    return n.element

def replace(self, p, e):
    n = self._validate(p)
    old = n.element
    n.element = e
    return old

```

3. Dynamic Array + Copying

12. Assignment / shallow / deep copy

```
import copy

india = [['Virat', 'Gill', 'Pant'], ['Bumrah', 'Siraj', 'Shami']]
team2 = india
team3 = copy.copy(india)
team4 = copy.deepcopy(india)

# team2: same outer object
# team3: new outer object, same inner lists
# team4: independent nested objects
```

13. Dynamic array

```
import ctypes

class SquadArray:
    def __init__(self):
        self.size = 0
        self.capacity = 1
        self.data = (ctypes.py_object * self.capacity)()

    def __len__(self):
        return self.size

    def resize(self, new_capacity):
        new_data = (ctypes.py_object * new_capacity)()
        for i in range(self.size):
            new_data[i] = self.data[i]
        self.data = new_data
        self.capacity = new_capacity

    def append(self, x):
        if self.size == self.capacity:
            self.resize(2 * self.capacity)
        self.data[self.size] = x
        self.size += 1

    def __getitem__(self, i):
        if i < 0 or i >= self.size:
            raise IndexError('invalid index')
        return self.data[i]

    def remove(self, name):
        pos = -1
        for i in range(self.size):
            if self.data[i].name == name:
                pos = i
                break
        if pos == -1:
            return
        for i in range(pos, self.size - 1):
            self.data[i] = self.data[i + 1]
        self.size -= 1

    def display(self):
        for i in range(self.size):
            print(self.data[i].name)
```

4. Binary Trees + BST

14. Binary tree traversals

```
class BNode:
    def __init__(self, key, left=None, right=None):
        self.key = key
        self.left = left
        self.right = right

def preorder(root):
    if root:
        print(root.key, end=' ')
        preorder(root.left)
        preorder(root.right)

def inorder(root):
    if root:
        inorder(root.left)
        print(root.key, end=' ')
        inorder(root.right)

def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.key, end=' ')
```

15. Reconstruct binary tree from preorder + inorder

```
def build_tree(pre, ino):
    if not pre:
        return None

    root = BNode(pre[0])
    k = ino.index(pre[0])

    left_in = ino[:k]
    right_in = ino[k+1:]
    left_size = len(left_in)

    root.left = build_tree(pre[1:left_size+1], left_in)
    root.right = build_tree(pre[left_size+1:], right_in)
    return root
```

16. BST insertion + search

```
class BSTNode:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None

def insert(root, key):
    if root is None:
        return BSTNode(key)
    if key < root.key:
        root.left = insert(root.left, key)
    elif key > root.key:
        root.right = insert(root.right, key)
    return root

def search(root, key):
    if root is None or root.key == key:
        return root
    if key < root.key:
        return search(root.left, key)
    return search(root.right, key)
```

17. BST deletion — three cases

```
def min_node(root):
    while root.left:
        root = root.left
    return root

def delete(root, key):
    if root is None:
        return None

    if key < root.key:
        root.left = delete(root.left, key)
```

```
elif key > root.key:
    root.right = delete(root.right, key)
else:
    if root.left is None:
        return root.right
    if root.right is None:
        return root.left

    s = min_node(root.right)
    root.key = s.key
    root.right = delete(root.right, s.key)

return root
```

5. AVL Trees

18. AVL height + balance + rotations

```
class AVLNode:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1

def height(n):
    return n.height if n else 0

def update(n):
    n.height = 1 + max(height(n.left), height(n.right))

def balance(n):
    return height(n.left) - height(n.right) if n else 0

def rotate_right(z):
    y = z.left
    t = y.right
    y.right = z
    z.left = t
    update(z)
    update(y)
    return y

def rotate_left(z):
    y = z.right
    t = y.left
    y.left = z
    z.right = t
    update(z)
    update(y)
    return y
```

19. AVL insertion

```
def avl_insert(root, key):
    if root is None:
        return AVLNode(key)

    if key < root.key:
        root.left = avl_insert(root.left, key)
    elif key > root.key:
        root.right = avl_insert(root.right, key)
    else:
        return root

    update(root)
    b = balance(root)

    if b > 1 and key < root.left.key:      # LL
        return rotate_right(root)
    if b < -1 and key > root.right.key:     # RR
        return rotate_left(root)
    if b > 1 and key > root.left.key:       # LR
        root.left = rotate_left(root.left)
        return rotate_right(root)
    if b < -1 and key < root.right.key:     # RL
        root.right = rotate_right(root.right)
        return rotate_left(root)

    return root
```

20. AVL deletion + rebalance

```
def avl_delete(root, key):
    if root is None:
        return None

    if key < root.key:
        root.left = avl_delete(root.left, key)
    elif key > root.key:
        root.right = avl_delete(root.right, key)
    else:
        if root.left is None:
            return root.right
        if root.right is None:
            return root.left
```

```

    s = root.right
    while s.left:
        s = s.left
    root.key = s.key
    root.right = avl_delete(root.right, s.key)

update(root)
b = balance(root)

if b > 1 and balance(root.left) >= 0:
    return rotate_right(root)
if b > 1 and balance(root.left) < 0:
    root.left = rotate_left(root.left)
    return rotate_right(root)
if b < -1 and balance(root.right) <= 0:
    return rotate_left(root)
if b < -1 and balance(root.right) > 0:
    root.right = rotate_right(root.right)
    return rotate_left(root)

return root

```

6. Min-Heap

21. MinHeap index formulas + insert

```
class MinHeap:
    def __init__(self):
        self.data = []

    def parent(self, i):
        return (i - 1) // 2

    def left(self, i):
        return 2*i + 1

    def right(self, i):
        return 2*i + 2

    def heapify_up(self, i):
        while i > 0:
            p = self.parent(i)
            if self.data[p] <= self.data[i]:
                break
            self.data[p], self.data[i] = self.data[i], self.data[p]
            i = p

    def insert(self, key):
        self.data.append(key)
        self.heapify_up(len(self.data)-1)
```

22. heapify down + extract min

```
def heapify_down(self, i):
    n = len(self.data)
    while True:
        l = self.left(i)
        r = self.right(i)
        small = i

        if l < n and self.data[l] < self.data[small]:
            small = l
        if r < n and self.data[r] < self.data[small]:
            small = r

        if small == i:
            break

        self.data[i], self.data[small] = self.data[small], self.data[i]
        i = small

def extract_min(self):
    if not self.data:
        raise IndexError('Heap is empty')

    root = self.data[0]
    last = self.data.pop()

    if self.data:
        self.data[0] = last
        self.heapify_down(0)

    return root

MinHeap.heapify_down = heapify_down
MinHeap.extract_min = extract_min
```

23. Heap delete(key)

```
def delete(self, key):
    try:
        i = self.data.index(key)
    except ValueError:
        return

    last = self.data.pop()
    if i == len(self.data):
        return

    self.data[i] = last

    if i > 0 and self.data[i] < self.data[self.parent(i)]:
        self.heapify_up(i)
    else:
```

```
        self.heapify_down(i)
MinHeap.delete = delete
```

7. Dynamic-array + complexity + exam patterns

24. Complexity you should be able to state immediately

```
Stack (Python list, top at end):
    push O(1)
    pop O(1)
    top O(1)

Queue (simple list + pop(0)):
    enqueue O(1)
    dequeue O(n)
    first O(1)

Singly linked list:
    add_first O(1)
    add_last O(n) without tail
    search O(n)
    delete by value O(n)

Doubly linked list:
    add_first/add_last O(1) with head+tail
    delete known node O(1)
    search O(n)

BST:
    average search/insert/delete O(log n)
    worst case O(n)

AVL:
    search/insert/delete O(log n)

Heap:
    peek O(1)
    insert O(log n)
    extract_min O(log n)
    delete in this lab O(n) because key search is O(n)

Dynamic array:
    normal append O(1)
    resize append O(n)
    amortized append with doubling O(1)
```

25. Two classic loops

```
# O(n)
for i in range(n):
    print(i)

# O(n^2)
for i in range(n):
    for j in range(n):
        print(i, j)

# O(n^2), because 0+1+...+(n-1)
for i in range(n):
    for j in range(i):
        print(i, j)

# O(log n)
i = 1
while i < n:
    i *= 2
```

26. Count equal pairs

```
def count_pairs(A):
    count = 0
    n = len(A)

    for i in range(n):
        for j in range(i+1, n):
            if A[i] == A[j]:
                count += 1

    return count

# Time: O(n^2)
# Extra space: O(1)
```

27. Two-sum dictionary pattern

```
def two_sum(A, target):
    seen = {}

    for i, x in enumerate(A):
        need = target - x
        if need in seen:
            return (seen[need], i)
        seen[x] = i

    return None

# Average time O(n), extra space O(n)
```

28. Frequency table

```
def frequency_table(A):
    freq = {}
    for x in A:
        if x not in freq:
            freq[x] = 0
        freq[x] += 1
    return freq
```

29. First non-repeating character

```
def first_non_repeating(s):
    freq = {}
    for ch in s:
        freq[ch] = freq.get(ch, 0) + 1

    for ch in s:
        if freq[ch] == 1:
            return ch

    return None
```

30. Reverse list without reverse()

```
def reverse_list(A):
    result = []
    for i in range(len(A)-1, -1, -1):
        result.append(A[i])
    return result
```

FINAL 90-MINUTE REHEARSAL

Before the exam, type these from memory in one blank Jupyter notebook:

1. Stack class
2. Queue class
3. balanced_brackets
4. SinglyLinkedList add_first/add_last/delete
5. reverse_linked_list
6. DoublyLinkedList add/delete
7. PositionalList add_before/add_after/delete
8. BST insert/search/delete
9. AVL rotate_left/rotate_right + insert
10. MinHeap insert/heapify_up
11. heapify_down/extract_min
12. Dynamic array resize/append
13. Write the complexity of each without looking

If you get stuck:

read the relevant solution -> close PDF -> type it yourself -> run tests -> explain each line aloud.

Memory rules

Singly delete:	prev.next = cur.next
Singly reverse:	nxt=cur.next; cur.next=prev; prev=cur; cur=nxt
Doubly delete:	pred.next=succ AND succ.prev=pred
BST:	compare -> left/right
BST 2 children:	successor = minimum of right subtree
AVL:	update height -> balance -> rotate
Heap insert:	append -> heapify_up
Heap extract:	save root -> last to root -> heapify_down
Dynamic array:	full -> 2x capacity -> copy -> append